

Lektion 11

OOP + Klaser

Mål

Efter endt undervisning kan den studerende:

Kende forskel på de 4 søjler indenfor OOP.

Lave simple klasser.

Kende forskel på instancer, objekter, klasser, metoder, attributter, funktioner osv.

Agenda

Mermaid

OOP

C# klasser

Opgaver

Næste gang

Mermaid Syntax

type	syntax
Synkron	->>
asynkron	-)
response	->>
destruction	-x
alternative	alt & else & end
loop	loop & end
optional	opt & end
parallel	par & and & end

Code & Diagram Side by Side

Code:

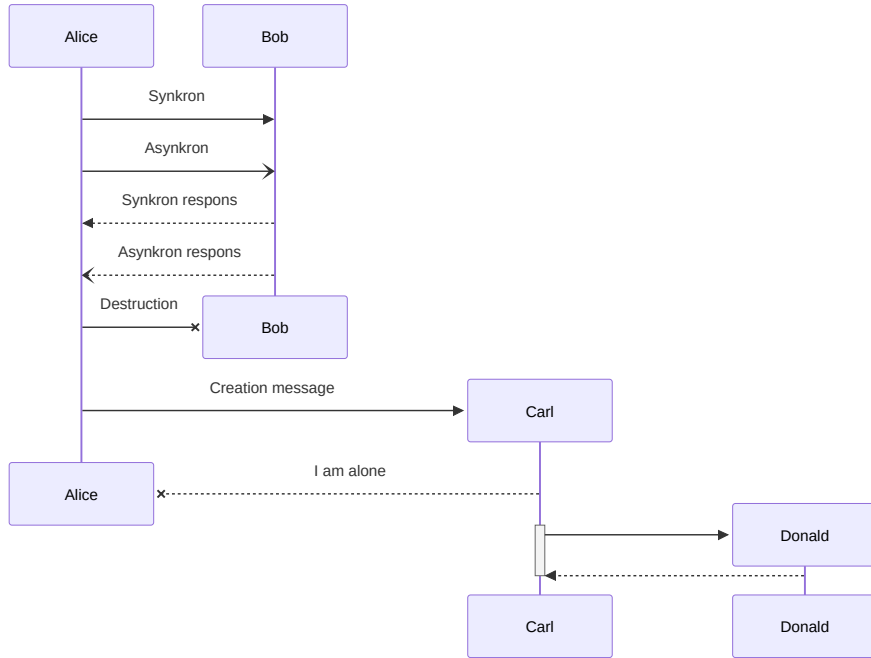
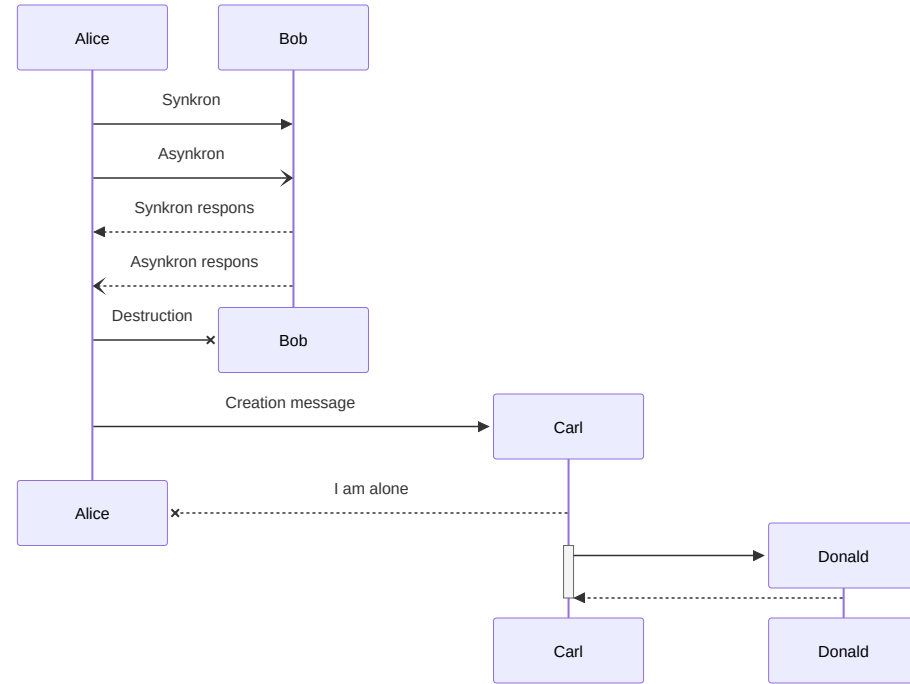
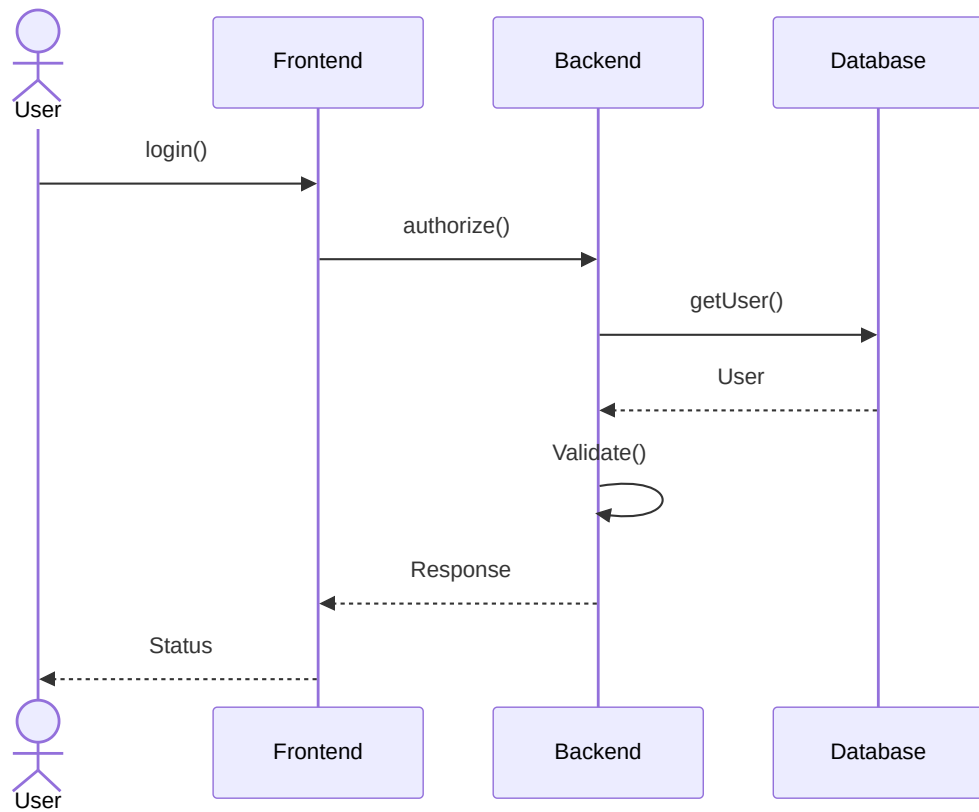
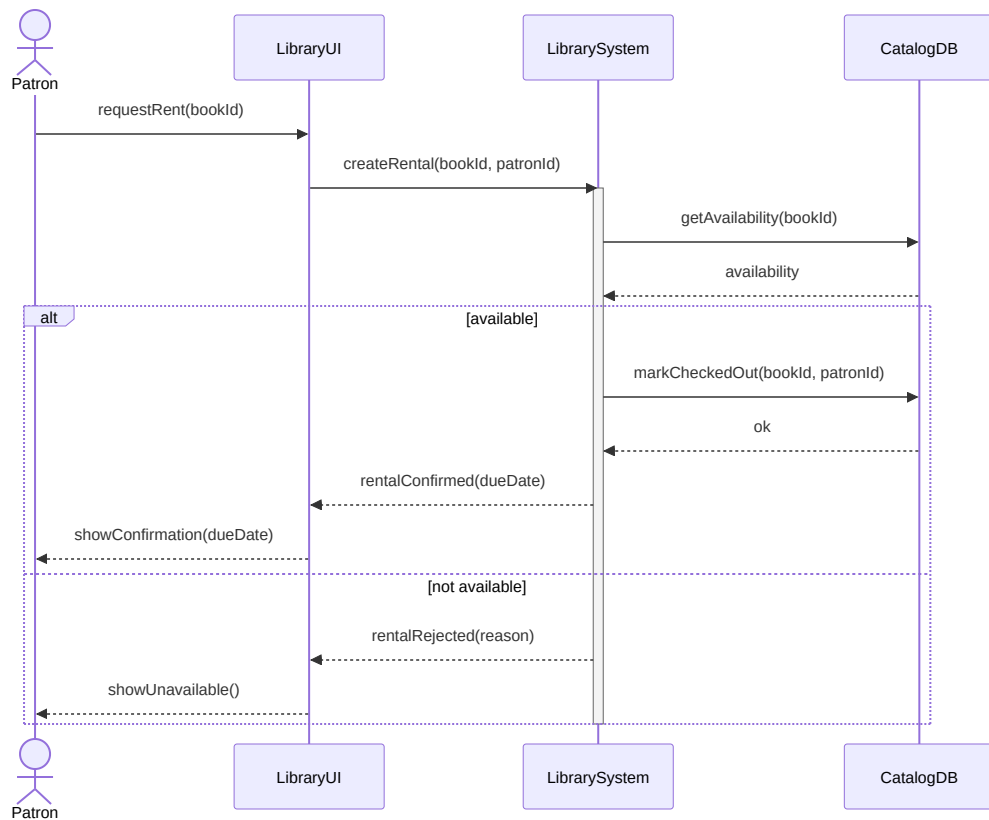


Diagram:







Nedarvning

Polymorfisme

Abstraktion

Enkapsulering

Nedarvning

Dyr kan sige noget \ Hund er et dyr - derfor kan
hund sige noget

Polymorfisme

Både hun og kat er et dyr. Hvis dyr skal gå, skal hund og kat gå.

Abstraktion

Dyr er en overordnet udgave af hund.

OOP - Enpsulering

Enkapsulering

Beskyt attributter.

public

private

protected

internal

C# klasser

```
class Animal // Class
{
    /*
        Attributes
    */
    protected string _sound; // Encapsulation

    public Animal() // Constructor
    { }
    /*
        Method
    */
    public string sound()
    {
        return _sound;
    }
    public void setSound(string newSound)
    {
        _sound = newSound;
    }
}
```

C# klasser - forsat

```
class Dog : Animal // Abstraction
{
    private string _name = "";

    public Dog(string name) // Constructor
    {
        _sound = "woof";
        _name = name;
    }
    /*
        Method
    */
    public string sound()
    {
        return _sound;
    }
}
```

C# klasser - forsat 2

```
Dog hansi /* object */= new Dog(); // Initiate class
```

```
List<Animal> animals = new List<Animal>  
{  
    new Dog("Buddy"),  
    new Cat("Misty"),  
    new Dog("Rex"),  
    new Cat("Luna")  
};
```

```
foreach (animal in animals)  
{  
    console.WriteLine(animal.sound())  
}
```


Opgave 1 – Klasser

Mål: Indkapsling + grundlæggende klasseforståelse.

Opret en klasse `BankAccount` med:

private felter: `_ownerName` , `_balance`

public properties (read-only for balance): `OwnerName` , `Balance`

Opret en constructor, der tager `ownerName` og start-beløb.

Opret metoder:

`Deposit(decimal amount)`

`Withdraw(decimal amount)` (må ikke tillade negative beløb eller overtræk)

Skriv en lille `Main` der, opretter en konto, indsætter penge, forsøger at hæve, for mange penge. Husk og print alle skridt så vi kan se hvad der forgår.

Opgave 2 – Abstraktion

Mål: Introduktion til abstraktion og nedarvning.

Lav en abstrakt baseklasse `Shape` med:

indkapslet felt `_name` + public property `Name`

abstrakt metode `double Area()`

Lav to klasser, der nedarver fra `Shape` :

`Circle` (med radius)

`Rectangle` (med width og height)

Sørg for at input valideres (fx radius > 0) i constructor (indkapsling/robusthed).

I `Main` , opret mindst én af hver og print navn + areal.

Opgave 3 – Polymorfisme med collections

Mål: Polymorfisme i praksis.

Udvid programmet fra opgave 2:

Opret `List<Shape> shapes`

Tilføj både `Circle` og `Rectangle` til listen

Skriv en metode `PrintReport(List<Shape> shapes)` der:

looper gennem alle shapes

printer `Name` og `Area()`

til sidst printer samlet areal (sum)

Tilføj en ny klasse `Triangle : Shape` med tre sider eller base+height (vælg én) og implementér `Area()` .

Næste gang

Klasse diagram. Vi skal snakke relationer, og pile (hint det bliver svært)

Vi laver også masser af øvelser indenfor OOP i C#. Så skal vi snakke C4 diagrammer, vi introducere begrebet og kigger på de 4 typer af diagrammer.